Enhancing Software Engineering with Fidelity-Based Prototyping

Lau Yee Wen
Department of Computer Science
on Data Engineering
Faculty of Computing
Universiti Teknologi Malaysia
Johor, Malaysia
lauyeewen@graduate.utm.my

Farra Nurzahin
Department of Computer Science
on Data Engineering
Faculty of Computing
Universiti Teknologi Malaysia
Johor, Malaysia
farranurzahin@graduate.utm.my

***Abstract*— Prototyping is essential to product development in various industries and provides a number of advantages, including user feedback, design evaluation and functionality testing. This paper explores many different kinds of prototype methodologies such as throwaway, evolutionary, incremental, low-fidelity and high-fidelity approaches. Additionally, this paper emphasizes prototypes effectiveness and functionality in software development and industrial design. By comparing different prototypes methods, the study evaluates their benefits, such as allowing feedback and effective communication, while also keep in mind the potential disadvantages. This comparative analysis offers guidance on choosing suitable prototyping techniques based on project specifications.**

Keywords*— *Prototyping, throwaway prototyping, evolutionary prototyping, incremental prototyping, low-fidelity, high-fidelity

I. INTRODUCTION

Prototyping is essential in the development and enhancement of products across a range of industries, such as software development, engineering, and design. It enables designers and engineers to produce early versions of a product, resulting in either a tangible or digital model that can be tested for functionality, usability, and design prior to final production. This iterative process minimizes risks, boosts user feedback, and ensures that the final product aligns with its intended objectives.

In today's fast-paced development environments, prototyping has become a vital step for reducing time-to-market and fostering innovation. Whether in software development, where wireframes and interactive models facilitate app creation, or in industrial design, where physical prototypes allow for testing of mechanical functions, prototyping offers a significant advantage in anticipating performance and identifying flaws early on.

This paper will delve into the various types of prototypes utilized across different sectors, the purpose of prototyping, and the related benefits and drawbacks. By exploring these elements, we aim to emphasize the crucial role that prototyping plays in driving innovation while also addressing the challenges associated with its implementation.

II. PURPOSE OF PROTOTYPING

Prototypes has been beneficial in some categories in software engineering processes. Firstly, the purpose of prototyping in software development approaches have shown to be able to lessen the total of rework required and help control thy of incomplete needs as they dynamically act to changes in user requirements [1]. This industry has considered several techniques to create prototypes as si or as partial implementations of systems and also to utilize them for a variety of different purposes. For example, prototypes are used to test the feasibility of some part of the system's technical aspects, or to decide the user requirements as specification tools [1]. These processes can motivate the efficiency of application development by breaking a complex problem into simpler chunks [1]. From [1], we include the process of testing the user satisfaction used in prototyping of software development approaches in Figure 1.

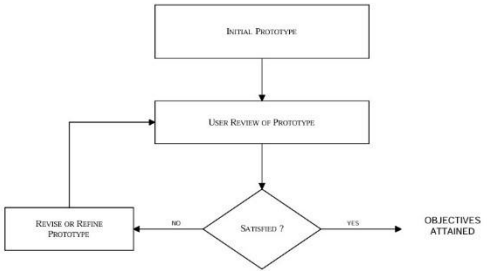

Figure 1

On the other hand, the term prototype was declared in [2] that it is commonly used in software engineering, human computer interaction (HCI) and design fields to highlight a specific part of a product used in the design process. Here we are about to discuss on the purpose of prototyping in design fields. Fundamentally, prototypes allow designers to naturally gain knowledge, discover, produce and refine designs [2]. Based on a variety of prototyping methods of usage in design phase from [2], they conclude that the purpose of prototype in design phases is to locate and generate valuable perspective in certain parts of the final design that were observed. Additionally, prototypes are intentionally produced objects that represent the design concepts. [2] argued that the prototypes are mainly be grouped for the following purposes: (1) testing and assessment; (2) understanding of user wants, values and experience; (3) idea production; and (4) designer communication. Each prototype can be utilized for several

purposes and not meant to be mutually exclusive to these categories.

III. Types of Prototype

There are various types of prototypes available, each offering distinct advantages and disadvantages. The choice of prototype depends on the specific objectives and requirements of the project, as well as the current stage of product development.

A. Throwaway Prototyping (Rapid Prototyping)

Throwaway or Rapid Prototyping refers to the creation of a model that will be discarded rather than incorporated into the final software product. After the preliminary requirements gathering, a simple working model of the system is constructed to visually show the users what their requirements may look like when they are implemented into a completed system [3].

The primary advantage of utilizing Throwaway Prototyping is that it facilitates rapid development and enables early feedback collection. The ability to design interfaces that users can actively test is another benefit of throwaway prototyping. It is much simpler for the user to comprehend how the system will function when it is visually displayed to them. The user interface is what the user sees as the system.

Paper prototyping is one technique for producing a low fidelity throwaway prototype. Since the prototype is made with paper and pencil, it functions similarly to the real product but doesn't look anything like it. Using a GUI Builder to create a click dummy—a prototype that mimics the goal system but lacks functionality—is another simple way to construct high fidelity throwaway prototypes.

B. Evolutionary Prototyping

Evolutionary Prototyping involves the continuous refinement of a prototype until it transforms into the final system. This approach is particularly advantageous when system requirements are not completely clear, as it enables developers to enhance the product iteratively based on ongoing feedback [4]. Evolutionary prototyping recognizes that we may not fully grasp all requirements and focuses on developing only those that are clearly understood. This approach enables the development team to incorporate features or implement changes that were not anticipated during the requirements and design stages.

Evolutionary prototyping has an advantage over Throwaway prototyping because it produces functional systems. While these prototypes may lack some of the features users envisioned, they can serve as interim solutions until the final system is completed. It guarantees that the end product closely meets user requirements.

Evolutionary Prototyping methodology is demonstrated by using the free version of a commercially available server-side scripting system along with HTML, Javascript, and MS Access. T [5].

C. Incremental prototyping

Incremental prototyping consists of breaking the system down into smaller components, with each one being prototyped separately. Once each part is developed and validated, they are combined to create the complete system. The goal of incremental prototyping is to enhance the design and deepen comprehension while making minimal changes to the overall structure. Incremental prototyping allows for a more profound comprehension and ultimately validates the design—while maintaining the overall design direction. [6]

Incremental prototyping enables the individual development and testing of critical components within a system. This strategy prioritizes and delivers essential functionalities early in the development phase. It ensures that vital modules are operational and can gather feedback while other parts are still being worked on. Moreover, by dividing the system into smaller sections, it simplifies the development process, making it more manageable to integrate each component effectively.

D. Low-Fidelity Prototyping

Low-fidelity prototypes are straightforward, quickly crafted models that emphasize structure and flow rather than intricate design details. Typically, these prototypes are hand-drawn or made using basic digital tools.

Lo-fi prototyping is praised for being both cost-effective and rapid to produce. It enables teams to experiment with early-stage design ideas without requiring detailed visuals or complex interactions. Due to their ease of modification, these prototypes encourage frequent iterations and foster discussions with stakeholders during the initial design phases. This method is especially beneficial when concentrating on the overall layout, navigation, and functionality of the system.

E. High-Fidelity Prototyping

High-fidelity prototypes, on the other hand, are complex, interactive models that closely resemble the finished product in terms of both functionality and appearance. The precise visuals of these prototypes include sophisticated interactions like animations and transitions, as well as color schemes and typography.

Although creating hi-fi prototypes requires more time and resources, they offer a more accurate representation of the user experience. Users can engage with these prototypes, providing in-depth feedback on both the visual design and functionality. High-fidelity prototypes are particularly valuable in the later stages of the design process, where thorough user testing is essential before proceeding to full development.

F. Low-Fidelity vs. High-Fidelity Prototyping

Low-fidelity prototypes are best suited for the **initial phases** of software engineering projects, where teams aim to explore broad design concepts and collect preliminary feedback. Their simplicity allows for quick modifications, enabling teams to swiftly iterate on various ideas. In contrast, high-fidelity prototypes are utilized in the **final phases** of design,

where in-depth user testing is essential to enhance the product's interaction design and visual appeal.

Aspect	***Low-Fidelity Prototyping***	***High-Fidelity Prototyping***
Detail	Basic layout, minimal visuals	Detail design, close to final
Creation Time	Quick, low time investment	Time−intensive, requires more effort
Cost	Inexpensive	Higher cost, due to specialized tools
Feedback	Early-stage feedback on structure	Late-stage feedback on aesthetics and usability
Modification	Easily modifiable	Focuses on user experience and aesthetics
Tools	Paper, whiteboards, or basic software	Specialized design tools

IV. COMPARATIVE ANALYSIS

Author(s), years and citation	Proposed solution	Description
J. Rudd, K. Stern, S. Isensee. (1996) [7]	Discussed on low and high-fidelity approach	- Focusing on their roles in user-interface process - Highlights on the difference between both types. - Limited empirical data from case studies or real-world applications.
T. Guimaraes. (1990) [8]	Prototyping: Orchestrating for success	- Explores on how well-executed prototyping. - Lacks specific examples and case studies. - Focused on prototyping in general.
M. Ryan, A. Doubleday. (1998) [9]	Throwaway Prototyping: Evaluation for requirements capture	- Investigates the value of early-stage in gathering. requirements - Limited hands-on interaction and feedback depth.
F. Kordon, J. Henkel. (2003) [10]	Rapid System Prototyping: An overview	- Explores rapid prototyping techniques for system development. - Narrow focus on advanced tools. - Limited coverage of prototyping stages.

The paper "Low vs. High-Fidelity Prototyping Debate" by J. Rudd, K. Stern & S. Isensee (1996) discusses on low-fidelity and high-fidelity approach with a focus on their roles in user-interfaces process. The paper also highlights on the differences between both prototypes approach. However, the paper is limited empirical data from case studies or real-world applications. Additionally, this paper also not discussed on the effectiveness of each prototype across varied settings.

T. Guimaraes (1990) introduced the "Prototyping: Orchestrating for Success", emphasizing on how well-executed on the usage of prototyping. However, the paper has lacks of understanding comparisons including the recommendations of the usage of different prototype on different prototype, mainly focuses on prototype in general, and absence of comparative analysis in each prototype.

Besides, M. Ryan, A. Doubleday (1998) introduce of the evaluation of throwaway prototyping, focuses on the investigation of the value of early-stage using a throwaway prototyping in gathering requirements. Throwaway prototype demonstrates to capture user preferences before final designs, improving user engagement, usability and project alignment with user needs. However, the paper has limited hands-on interaction, limited insights on technical implementation. and provide no feedback depth from the user experience.

Lastly, the paper "Rapid System Prototyping: An overview" by F. Kordon, J. Henkel. (2003) explores rapid prototyping techniques for system development. It highlights methods like throw-away, incremental and evolutionary prototyping to improve design exploration and validate early requirements. However, this paper has limited coverage of prototyping stages, provides insufficient guidance on choosing appropriate methods and has narrow focus on advanced tools.

In summary, the comparative analysis of the four studies reveals various limitations in their discussion of prototyping approaches. Prototyping in general, including throwaway prototypes and other rapid system prototyping have offered unique approaches to system development problem, while [10] highlights the comparison between low-fidelity and high-fidelity prototypes. However, all four papers have lack of comprehensive comparisons and recommendations for various prototype kinds, insufficient observation technical implementation and user experience feedback, insufficient recommendations for choosing suitable techniques and empirical evidence to evaluate the efficiency of low and high-fidelity methods in various settings. The project's particular requirements should be taken into account while choosing a methodology or approach

V. ADVANTAGES & DISADVANTAGES OF PROTOTYPE

Prototyping presents several significant benefits in software development. It enables developers to obtain user feedback early in the process, ensuring that the system meets user needs. Prototyping can be viewed as "an aid to building the right software by having multiple attempts and learning from errors."[11] Users perceive information systems more positively and experience greater satisfaction when a prototyping approach is implemented. This sense of user satisfaction stems from the active engagement of users that the prototyping method necessitates.

Additionally, prototypes enhance communication between developers and stakeholders, as visual representations are more comprehensible than written specifications. Thus, errors and misunderstandings can be caught early, leading to a more refined final product. Prototyping also speeds up the development process, enabling quick iterations and modifications based on user feedback through functional prototypes.

On the flip side, some researchers have identified possible risks associated with prototyping, and it can result in inflated expectations [12]. One major challenge is scope creep, which occurs when users continuously request changes and enhancements, causing the project to expand indefinitely. This can lead to delays and increased costs as the development process becomes less organized. Moreover, users might confuse the prototype with the final product, creating unrealistic expectations regarding the time and effort needed to complete the finished system. In certain instances, an overemphasis on perfecting prototypes may result in insufficient attention to proper system documentation.

However, despite these challenges, prototyping can provide significant benefits when developing relatively unstructured systems that require high-quality user interfaces and executive information systems. It can increase productivity and improve the overall quality of the finished product.

VI. CONCLUSION

Prototyping is a flexible and powerful techniques that is crucial for improving product concepts and meeting consumer needs in early development. From the comparative analysis of variety prototyping approaches, it is clear that each one of types has special benefits that correspond with variety phases of the design and development process. Techniques such as throwaway prototyping allow for rapid exploration by dropping early form after feedback, evolutionary prototyping enable stages improvement by continuously builds upon the same model and incremental prototyping divided development into more feasible, smaller components that are gradually integrated and tested. While low-fidelity prototypes enable rapid exploration and evaluation gathering, high-fidelity structures offer a near-final representation perfect for user experience testing. Ultimately, prototyping methodologies should carefully be considered based on the particular project situation and the intended results. Moreover, a focus on an establishing a balance between precise functionality and detailed functionality need to keep in mind to maximise user satisfaction and project efficiency.

REFERENCES

[1] M. Carr and J. Verner, "Prototyping and software development approaches", *Department of Information Systems*, pp. 3-4, 1997.

[2] Y. Lim, E. Stolterman, J. Tenenberg, "The anatomy of Prototypes: Prototypes as Filters, Prototypes as Manifestations of Design Ideas", *ACM Transactions on Computer-Human Interaction (TOCHI)*, pp. 2-12, 2008.

[3] A. T. Sadabadi and N. M. Tabatabaei, "Rapid prototyping for software projects with user interfaces," Department of Computer Systems and Informatics Seraj Higher Education Institute, Iran, vol. 9, pp. 85-90, 2009.

[4] L. White, "Evolutionary Prototyping: A Flexible Approach," *Software Development Practices*, vol. 9, pp. 42-49, 2020.

[5] R. F. Zant, "Hands-on prototyping in system analysis and design," *Issues in Information Systems*, vol. 6, no. 1, pp. 10-14, 2005.

[6] G. Tate and J. Verner, "Case study of risk management, incremental development, and evolutionary prototyping," *Information and Software Technology*, vol. 32, no. 3, pp. 207-214, 1990.

[7] J. Rudd, K. Stern, S. Isensee, "Low vs. High-fidelity prototyping debate", vol.3, pp. 80-84, 1996.

[8] T. Guimaraes, "Prototyping: orchestrating for success", June, 2016. [Online serial]. Available: https://www.researchgate.net/profile/Tor-Guimaraes/publication/247290193_Prototyping_Orchestrating_for_success/links/575985db08ae414b8e43e61f/Prototyping-Orchestrating-for-success.pdf (Accessed Oct. 26, 2024).

[9] M. Ryan and A. Doubleday, "Evaluating 'Throw Away' Prototyping for Requirements Capture", pp.2-8, 1998.

[10] F. Kordon and J. Henkel "An overview of Rapid System Prototyping today", *Design automation for embedded systems*, pp. 4-8, 2003.

[11] N. Cerpa and J. Verner, "Prototyping: some new results," *Information and Software Technology*, vol. 38, no. 12, pp. 743-755, 1996.

[12] G. Tate and J. Verner, Case study of risk management, incremental development and evolutionary prototyping, Inf. and Soft. Technol., 32 (1990) 2077214.